

# Solar Tracking System – ADC, DMA, Timer, and Interrupt Analysis report

## 1. System Overview

### Project Summary

This project implements an embedded solar-tracking, light-responsive curtain system intended to reduce building cooling load by actively controlling sunlight exposure. Four photoresistors measure light intensity from different directions around the structure. The firmware continuously samples all sensors using a single ADC peripheral configured in scan mode with DMA support, and drives four stepper motors—one per curtain axis—to raise or lower curtain segments based on configurable light thresholds. A manual override pushbutton implemented using an external interrupt (EXTI) allows the user to command curtain motion regardless of ambient lighting conditions.

### System Behavior (Core Operating Concept)

At runtime, the microcontroller evaluates the four light sensor readings to determine directional sunlight intensity. When a sensor reading exceeds a defined “bright” threshold, the corresponding motor is commanded to move the curtain in the direction that blocks incoming sunlight. When the reading falls below a defined “dim” threshold, the motor reverses direction to reopen the curtain. The use of separate ON and OFF thresholds implements hysteresis, preventing rapid oscillation or motor chatter when light levels fluctuate near the decision boundary.

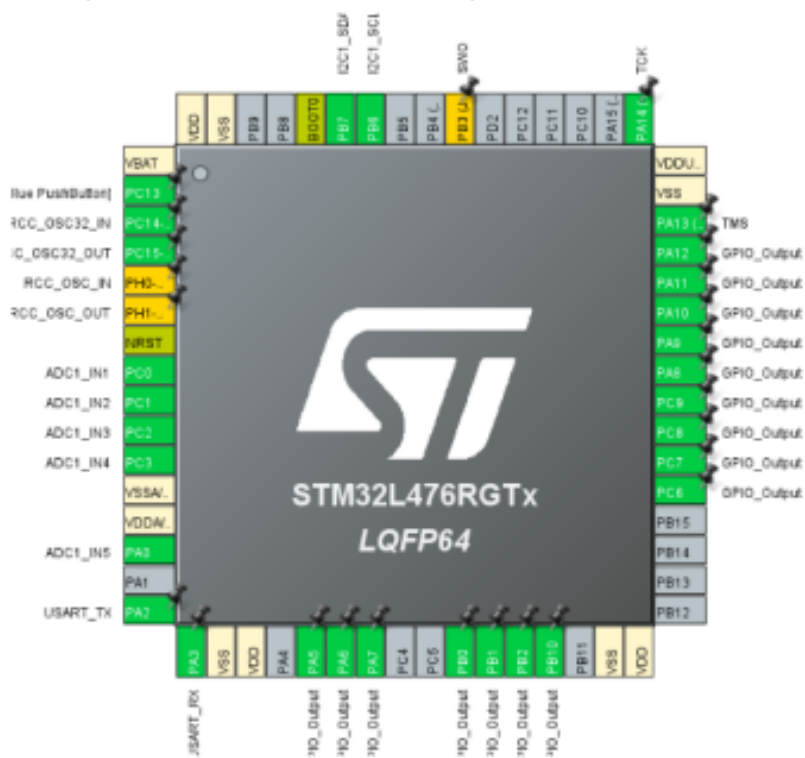
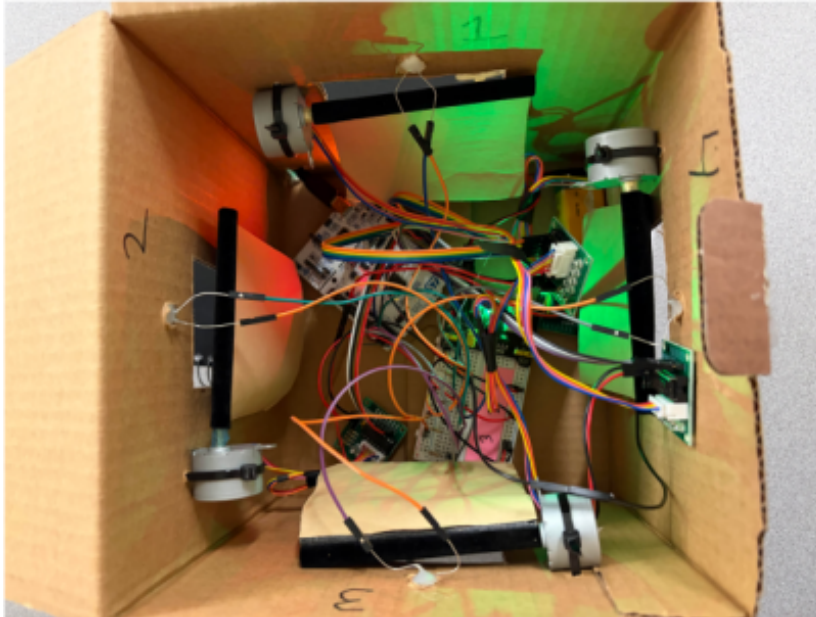
### Hardware Overview

The system is built around the STM32L476RG microcontroller and uses the following primary hardware components:

- STM32L476RG microcontroller utilizing ADC1, DMA, TIM2, and USART2 peripherals
- Four photoresistors connected to pins PC0–PC3 and configured as ADC1 channels 1–4
- Four stepper motors driven using four-wire GPIO interfaces mapped across GPIOA, GPIOB, and GPIOC

### Firmware Architecture

The firmware architecture divides functionality across dedicated hardware peripherals to maximize determinism and efficiency. ADC1 continuously samples all photoresistors, while DMA transfers conversion results directly into RAM without CPU intervention. TIM2 provides microsecond-resolution timing for stepper motor sequencing, ensuring repeatable motor motion. USART2 streams live sensor and state data for debugging and validation. This partitioned approach allows the sensing pipeline to operate continuously in the background while the CPU focuses on decision logic, motor control, and system supervision.



**Figure: STM Cube IDE Graphical Pinout**

STM32L476RG pin usage summary (ADC inputs on PC0–PC3 and motor GPIO outputs on PA/PB/PC).

## 2. ADC Channel Scanning and DMA Data Path

ADC1 is configured in scan conversion mode to sample four photoresistor inputs connected to pins PC0–PC3. Each input is mapped to a distinct ADC channel and assigned a specific rank, which defines the exact order in which conversions occur during a scan sequence. This rank-based configuration ensures that the conversion order is deterministic and repeatable, which is essential when associating each sensor reading with its corresponding curtain motor.

Using a single ADC with multiple channels reduces peripheral usage and simplifies clock configuration while guaranteeing that all sensor measurements are captured within a tightly bounded time window. This is particularly important for relative light comparison, as asynchronous sampling could lead to incorrect control decisions if lighting conditions change rapidly.

DMA is enabled to transfer ADC conversion results directly into a RAM buffer without CPU polling or interrupt-driven data handling. Once initialized, the ADC continuously converts the configured channel sequence, and after each conversion the DMA automatically copies the ADC data register into the next buffer element. This creates a continuous data stream where the buffer indices correspond directly to ADC ranks.

Continuous conversion mode combined with continuous DMA requests allows the ADC–DMA subsystem to operate autonomously throughout runtime. As a result, the CPU remains available for motor control, state tracking, and system supervision rather than being burdened with repetitive ADC management tasks. This design significantly reduces interrupt load and improves overall system scalability.

Code Excerpt 1: ADC1 scan sequence (ranks) defining buffer order for four photoresistor inputs.

```
/* ADC1 configured for scan + continuous + DMA */
hadc1.Init.ScanConvMode = ADC_SCAN_ENABLE;
hadc1.Init.ContinuousConvMode = ENABLE;
hadc1.Init.NbrOfConversion = 4;
hadc1.Init.EOCSelection = ADC_EOC_SEQ_CONV;
hadc1.Init.DMAContinuousRequests = ENABLE;

/* Rank 1: PC2 -> ADC1_IN3 */
sConfig.Channel = ADC_CHANNEL_3;
sConfig.Rank = ADC_REGULAR_RANK_1;
HAL_ADC_ConfigChannel(&hadc1, &sConfig);

/* Rank 2: PC3 -> ADC1_IN4 */
sConfig.Channel = ADC_CHANNEL_4;
sConfig.Rank = ADC_REGULAR_RANK_2;
HAL_ADC_ConfigChannel(&hadc1, &sConfig);

/* Rank 3: PC1 -> ADC1_IN2 */
sConfig.Channel = ADC_CHANNEL_2;
sConfig.Rank = ADC_REGULAR_RANK_3;
HAL_ADC_ConfigChannel(&hadc1, &sConfig);

/* Rank 4: PC0 -> ADC1_IN1 */
sConfig.Channel = ADC_CHANNEL_1;
sConfig.Rank = ADC_REGULAR_RANK_4;
HAL_ADC_ConfigChannel(&hadc1, &sConfig);
```

Code Excerpt 2: Starting ADC + DMA to stream scan results into a 4-sample RAM buffer.

```

/* ADC DMA buffer: adc[0..3] continuously refreshed by DMA */
volatile uint16_t adc[4] = {0, 0, 0, 0};

/* Start continuous conversions with DMA transfers into adc[] */
if (HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc, 4) != HAL_OK) {
    Error_Handler();
}

```

### 3. Control Logic (Thresholding + Hysteresis)

Each curtain quadrant is controlled independently using its corresponding light sensor. Light intensity data for all four quadrants is continuously acquired using ADC1 configured in scan mode and transferred into memory using Direct Memory Access (DMA). By enabling DMA, the system eliminates CPU involvement in the continuous acquisition of ADC data. Once the ADC scan sequence is initiated, each conversion result is automatically transferred from the ADC data register into a predefined RAM buffer without generating CPU interrupts or requiring register polling.

In this configuration, the DMA operates in circular mode, continuously refreshing a four-element buffer that corresponds directly to the ADC scan ranks. The memory layout is deterministic: the first buffer index always contains the conversion result of Rank 1, the second index Rank 2, and so forth. This tight coupling between ADC rank configuration and buffer indexing allows the firmware to directly map sensor readings to their corresponding motor control logic with no additional data manipulation or sorting. As a result, each curtain control loop can directly reference `adc[i]` for its associated photoresistor.

By offloading ADC data transfers to DMA, the CPU remains available for time-critical tasks such as stepper motor control, timing supervision, and state management. Continuous conversion mode combined with DMA continuous requests enables the ADC–DMA subsystem to function autonomously after initialization, forming a dedicated real-time sensing pipeline that reliably supplies up-to-date sensor data to the control logic.

Using the continuously updated ADC values, the firmware implements a two-threshold hysteresis scheme to determine curtain motion. An upper threshold (TH\_ON) and a lower threshold (TH\_OFF) define the transition points between curtain states. A curtain transitions to the lowered, sun-blocking state only when the corresponding sensor reading exceeds TH\_ON and remains in that state until the reading falls below TH\_OFF. This hysteresis approach prevents rapid oscillation or “chatter” when light levels fluctuate near a single threshold, such as during partial cloud cover or minor environmental changes.

A state variable (`atHigh[i]`) is used to track the current position of each curtain quadrant. Motor commands are issued only when a valid state transition occurs, ensuring that motors are not repeatedly actuated unnecessarily. This reduces mechanical wear, limits power consumption, and improves overall system stability.

Code Excerpt 3: Hysteresis-based decision logic for raising/lowering each curtain quadrant.

```

#define TH_ON 2000
#define TH_OFF 1700

if (!atHigh[i] && (adc[i] > TH_ON)) {
    motorStepCW(motors[i], BIG_STEPS, STEP_DELAY_US);
    motorStop(motors[i]);
    atHigh[i] = 1; // now "lowered"
}
else if (atHigh[i] && (adc[i] < TH_OFF)) {
    motorStepCCW(motors[i], BIG_STEPS, STEP_DELAY_US);
}

```

```

motorStop(motors[i]);
atHigh[i] = 0; // now "raised"
}

```

## 4. Timer (TIM2) for Microsecond-Level Motor Timing

Stepper motor control in this system requires precise and repeatable timing between successive coil energization steps. Timer 2 (TIM2) is configured as a free-running up-counter with a prescaler selected to produce approximately one microsecond per timer tick. This configuration allows the firmware to implement a deterministic microsecond-scale delay function (`microDelay`) that defines the dwell time between step transitions in the motor control sequence.

The stepper motors are driven using an eight-state half-step sequence, which provides smoother motion and improved positioning resolution compared to full-step operation. Between each state transition, the firmware invokes the timer-based delay function to ensure consistent timing across all steps. This guarantees uniform motor behavior regardless of instruction execution variability or compiler optimization effects.

Although the delay mechanism is implemented as a blocking loop for simplicity, it provides accurate and repeatable motor timing suitable for system validation and functional testing. From a system-level perspective, using a hardware timer for delay generation ensures that motor timing remains independent of software execution jitter, which is critical for predictable electromechanical behavior.

In future revisions, this timing logic could be migrated to a non-blocking, interrupt-driven state machine or timer callback architecture. Such an approach would preserve precise motor timing while improving CPU availability for additional control features, safety monitoring, or power-management tasks.

Code Excerpt 4: TIM2-based microsecond delay used to time stepper coil sequencing.

```

/* TIM2 configured for ~1 MHz tick (typical if timer clock ≈ 80 MHz) */
htim2.Init.Prescaler = 80 - 1;
htim2.Init.Period = 0xFFFFFFFF;
HAL_TIM_Base_Start(&htim2);

static void microDelay(uint16_t us) {
    __HAL_TIM_SET_COUNTER(&htim2, 0);
    while ( __HAL_TIM_GET_COUNTER(&htim2) < us) { }
}

```

## 5. Manual Override via External Interrupt

In addition to autonomous, sensor-driven operation, the system supports a manual override mode implemented using an external interrupt (EXTI). A pushbutton input is configured as an EXTI source, allowing the user to bypass automatic light-based control and directly command curtain motion. When the pushbutton is pressed, the associated interrupt service routine (ISR) sets a control flag that modifies system behavior within the main control loop.

This interrupt-driven approach ensures that user commands are handled with minimal latency and without interfering with ongoing ADC or DMA operations. Because sensor acquisition is performed continuously in the background by the ADC–DMA pipeline, the interrupt logic remains lightweight and does not disrupt real-time sensing. The main loop simply evaluates the override flag and conditionally bypasses the sensor-based

decision logic when manual control is active.

By separating manual override functionality from the sensor acquisition and control pipeline, the system maintains robustness and predictability when transitioning between automatic and manual modes. This design preserves deterministic background operation while providing a safe and responsive user control path, exemplifying good embedded system design practice where responsiveness and system stability are both prioritized.

Code Excerpt 5: Typical EXTI interrupt pattern for a manual override flag (adapt to the project button pin).

```
/* Example pattern (conceptual): */
volatile uint8_t manualOverride = 0;
volatile uint8_t forceCurtainsUp = 1;

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == GPIO_PIN_13) { // example button pin
        manualOverride ^= 1; // toggle auto/manual
    }
}

/* In main loop: */
if (manualOverride) {
    /* command motors to desired position (up/down) */
} else {
    /* normal ADC/DMA-based control */
}
```

## 6. GPIO Mapping Summary

Each stepper motor is driven by four GPIO outputs, with motors M1–M4 mapped across GPIOA, GPIOB, and GPIOC. ADC inputs are placed on PC0–PC3 and configured as analog pins. This grouping improves wiring clarity, simplifies debugging, and allows each quadrant to be tested or modified independently.

Code Excerpt 6: Pin and buffer mapping documentation used to keep wiring and firmware aligned.

```
/* Motor pin maps:
 * M1: PA5, PA6, PA7, PA8
 * M2: PA9, PA10, PA12, PA11
 * M3: PB2, PB1, PB10, PB0
 * M4: PC6, PC8, PC9, PC7
 *
 * ADC buffer order:
 * adc[0] = PC2 (ADC1_IN3)
 * adc[1] = PC3 (ADC1_IN4)
 * adc[2] = PC1 (ADC1_IN2)
 * adc[3] = PC0 (ADC1_IN1)
 */
```

## 7. Power Constraints and Low-Power Design Considerations

The Solar Tracking System was developed under strict power availability constraints, which significantly influenced architectural decisions. The use of multiple stepper motors, continuous ADC sampling, and real-time control logic inherently results in higher power consumption compared to simple sensing systems. As a result, power efficiency was considered a limiting factor during development, particularly for prolonged autonomous operation.

In the current implementation, the system prioritizes responsiveness and validation of control logic over aggressive power optimization. Continuous ADC conversion and frequent motor actuation ensure reliable sunlight tracking but increase average current draw. This tradeoff was accepted for development and testing purposes to ensure functional correctness before introducing power-saving optimizations.

A natural extension of the design is the incorporation of low-power operating modes. The STM32L476RG microcontroller supports multiple low-power states, such as Sleep and Stop modes, which could be leveraged to significantly reduce energy consumption during periods of minimal environmental change. In a future revision, the system could enter a low-power sleep state after curtain positioning is completed and wake periodically using a low-power timer or real-time clock (RTC).

Upon waking, the system would briefly enable the ADC and sample the photoresistors to evaluate ambient light conditions. If the measured light levels remain within the hysteresis band, the system could immediately return to sleep without motor actuation. Only when a meaningful change in sunlight is detected would the system fully wake, re-enable motor drivers, and adjust the curtain position accordingly.

This duty-cycled sensing approach would substantially reduce average power consumption while preserving system responsiveness. Such a design aligns with real-world embedded control systems that balance performance with energy efficiency, particularly in applications powered by limited or renewable energy sources.